

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/261465403>

A process model for Requirements Change Management in collocated software development

Conference Paper · October 2012

DOI: 10.1109/IS3e.2012.6414949

CITATIONS

10

READS

1,790

4 authors, including:



Arif Ali Khan

Nanjing University of Aeronautics & Astronautics

54 PUBLICATIONS 218 CITATIONS

[SEE PROFILE](#)



Shuib Basri

Universiti Teknologi PETRONAS

88 PUBLICATIONS 466 CITATIONS

[SEE PROFILE](#)



Fazal-e- Amin

King Saud University

47 PUBLICATIONS 196 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Global software engineering, Empirical software engineering [View project](#)



Risk Management [View project](#)

A Process model for Requirements Change Management in Collocated Software Development

Arif Ali Khan¹, Shuib Basri¹, P.D.D. Dominic¹ and Fazal-e-Amin²

Department of Computer and Information Sciences Universiti Teknologi PETRONAS

Bandar Seri Iskandar, Tronoh Perak, Malaysia¹

College of Computer Science and Information Studies, Government College University, Faisalabad, Pakistan²

arifaan_beghian@yahoo.com¹, shuib_basri@petronas.com.my¹, dhanapal_d@petronas.com.my¹

fazal.e.amin@gmail.com²

Abstract—Requirements Change Management (RCM) could occur at any phase of the software development life cycle. Therefore, RCM is considered to be a difficult task in software development organizations. The purpose of this study is to propose a model for RCM that will be implemented in collocated software development organizations. The model is based on the data collected from different RCM reports, models, research papers and different case studies related to RCM. The model has seven core stages, i.e., change request, validate, reject, batch, implement, verify and update. Propose model will eliminate the limitations of the available RCM models.

Index Terms—Model, Requirements Change Management, Process, Collocated Software Development

I. INTRODUCTION

Requirements change can occur during the software development life cycle. Management of these changes are a must to successfully develop a software system [1]. The objective of the requirements management is to identify, categorize, communicate and control the volatile requirements in the software system [2]. Requirements change is necessary in order to fulfill the needs of the customers [3]. RCM have been identified as a long standing problem in collocated software development organizations. In (2004) a report was published by Standish Group International in which they surveyed 13,522 software projects and only 39 percent of the software projects were successful, 18 percent failed and 53 percent were suspected. The main cause of the failed projects were the requirements change [4]. Even in huge software systems, the requirements change was noted up to 70% [5]. The RCM process is important in collocated and distributed software development [3]. McGee and Greer [6] mentioned different reasons for the requirements change which are: change in organizational strategies, requirements specification quality, technical complexities, period of requirements etc. The requirements change management process model is composed of different elements such as activities, roles and artifacts [3]. Due to an improper RCM, the complete failure of the system can occur or it can be the cause of business loss. RCM is a difficult job but proper RCM can be very worthwhile for software development organizations [7].

An effective requirements change management process model would help to complete the project in timely manner and within budget. Otherwise, a simple change could affect other requirements that would result in serious problems thereby the project failure might occurred [8]. Similarly a

proper RCM model would organize the requirements change process [9].

In this work, a process model has been proposed for RCM in collocated software development organizations. Collocated software development organizations are those organizations in which the software development team members work together at one place. In collocated software development, the organizations don't have distributed software development sites.

In this process model, all the stages of the RCM have been elaborated. Each of the stages has specific activities which should be performed during the RCM.

II. LITERATURE REVIEW

Bhatti et al. [3] proposed a change process consisting of six basic steps as shown in Fig. 1. In the first step, the change is initially requested. The second step is the receiving step in which the change request would be received for consideration. The change request is received using a specific change request form. After the change request has been received, the next step is to evaluate the change request by the change control board (CCB) to indicate the impact of the change on different aspects of the system. In the fourth step, the decision regarding the approval or rejection of the change request is made collectively by the CCB, quality assurance (QA) team, project team members and those who made the request for the change. The decision is based on the possibility of the change request. The entire accepted change request is sent forward to the implementation phase, where the approved changes are incorporated into the system. The last step is the configuration step in which the records of the entire change requests will be stored.

This process model covers the change management activities during the software development life cycle. However, the model is lacking some important activities such as verification and batch activity. Due to the unavailability of the verification activity, it is difficult to determine whether the change made in the system is working properly or not? Similarly in the absence of a batch activity, the model does not clearly represent those changes that are not implemented immediately but can be implemented in the future [9].

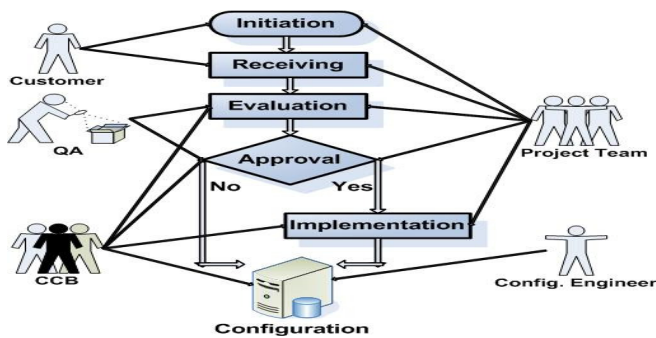


Figure 1. Bhatti et al. Change Management Process [3]

A spiral like change management model is presented in [10] which consists of four steps or cycles as shown in Fig. 2. In the first cycle; some amendments are requested in the form of the addition of some new features or corrections of the errors in the existing system. At the end of the first cycle, the

problem owner decides whether change is needed to be made or not and if it is necessary, then how it should be accommodated. The second step of the model occurs when the desired change needs to be looked at from non-technical view point. The third cycle is the planning step in which a plan is made for the implementation of the change. Implementation is the last step of the change request. In this step, the requested change is implemented and also the verification of the technical solution occurs in this step. After the implementation of the change, all of the updates about the change are recorded. This model [10] is a generic model and it is used to manage the requirements change in both the development and maintenance phases. However, this model does not provide any information regarding the change verification or batch activity [9].

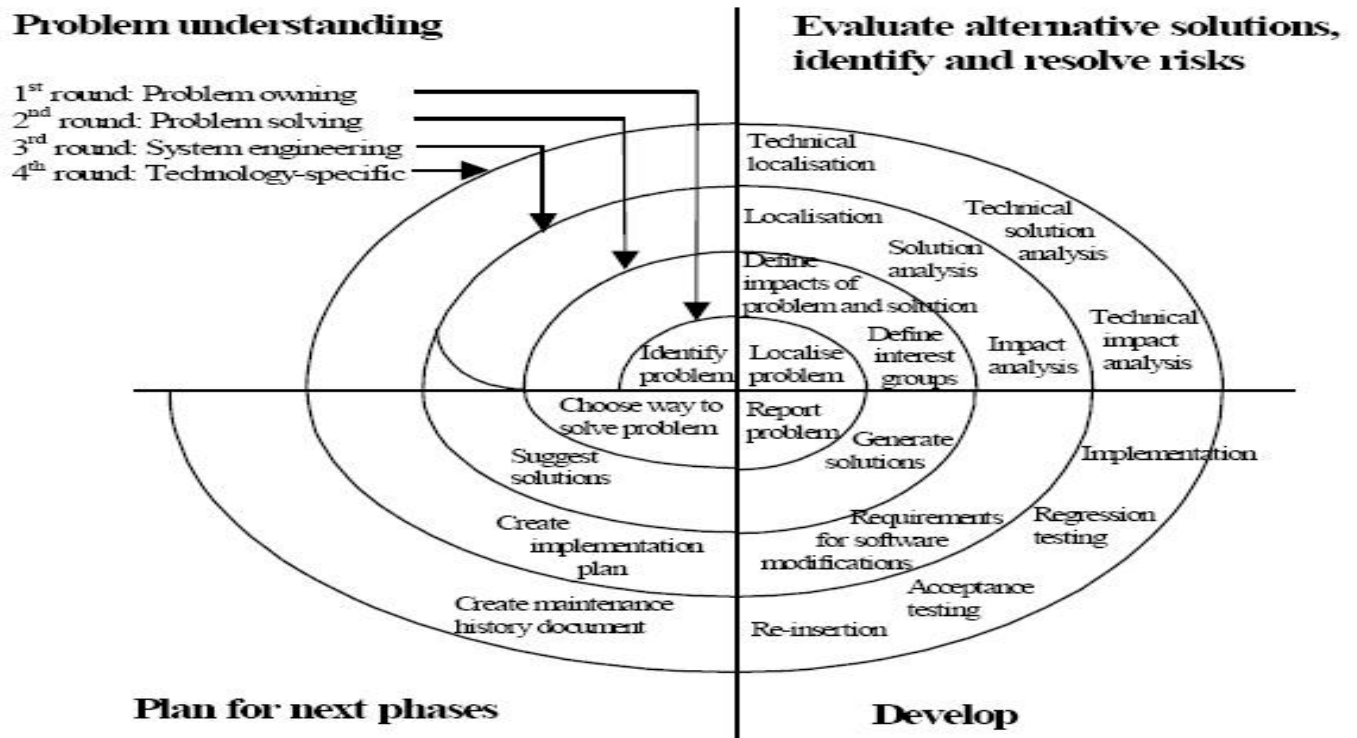


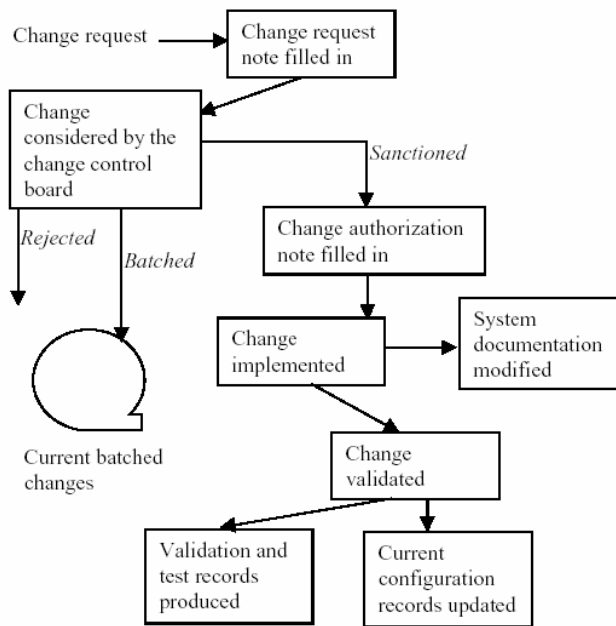
Figure 2. The spiral like change management model [10]

Ince's Model [10] begins with the change request from the stakeholders as shown in Fig. 3. The change request is then recorded on the change request note and later on forwarded to the CCB. The CCB makes decisions about the change request. After considering the change request, the CCB can make three types of decisions, i.e.,

- Reject the requested change (change cannot be implemented due to any reason).
- Batch the change request for some time (change would take place after some period of time).
- Accept the change and implement it immediately.

If the change request is accepted for immediate implementation, then the change authorization note would be filled. The next step is the change implementation. After implementation the system documents are updated. After the change implementation is performed, it is validated in the validation phase. Afterwards, the test records of all the

implemented changes are produced and the configuration records are modified. Then all the stakeholders are informed about the implemented change. Ince's Model [10] covers the requirements change management process; however, the model lacks the verification activity [9]. Therefore, it is difficult to ascertain whether the change made in the system is functioning accurately or not.



sources for the change are provided by the user who makes the change request. The requested changes are sent to the change manage phase in which the requested changes are managed by the change managers. After being accepting, the change request is forwarded to the implementation phase where these changes are implemented. The implemented changes are verified by testing the code and inspecting the papers. After the verification phase, the change managers are informed about the changes and who released the change in the system as shown in Fig.4. Olsen's change management model [11] points to the fact that change is a key element in the software development. However, the change request and the batch activities are not part of this model. The model does not provide any information about the change request such as who requested the change and, how and when the change was requested. Similarly, the batch activity is completely neglected in the model due to which the model is not given any idea about those changes which are not implemented immediately but can be implemented after some period of time [9].

Figure 3. Ince's change management model [10]

Similarly, Olsen's change management model [11] is applicable to both software and development phases. All the

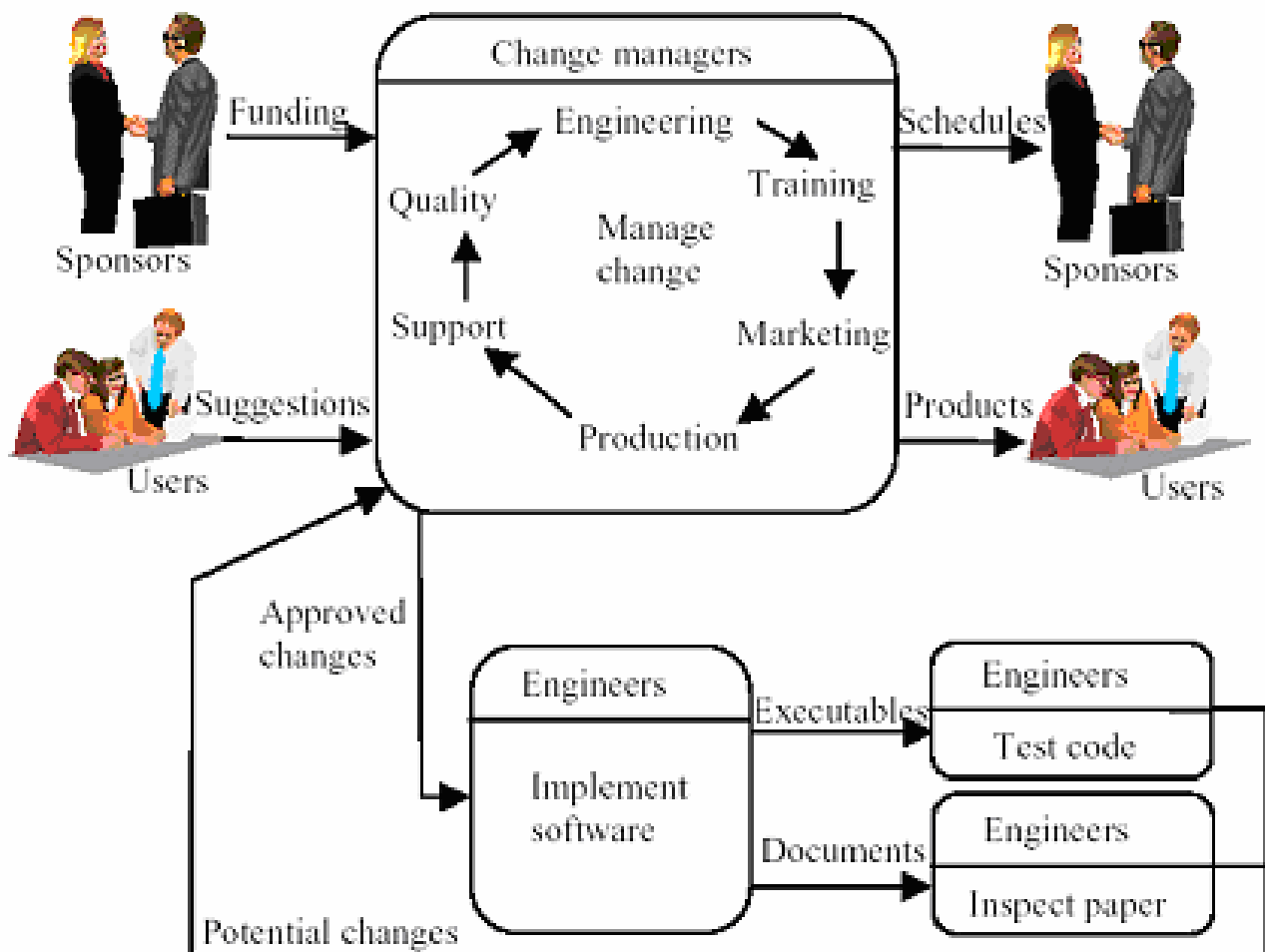


Figure 4. Olsen's change management model[11]

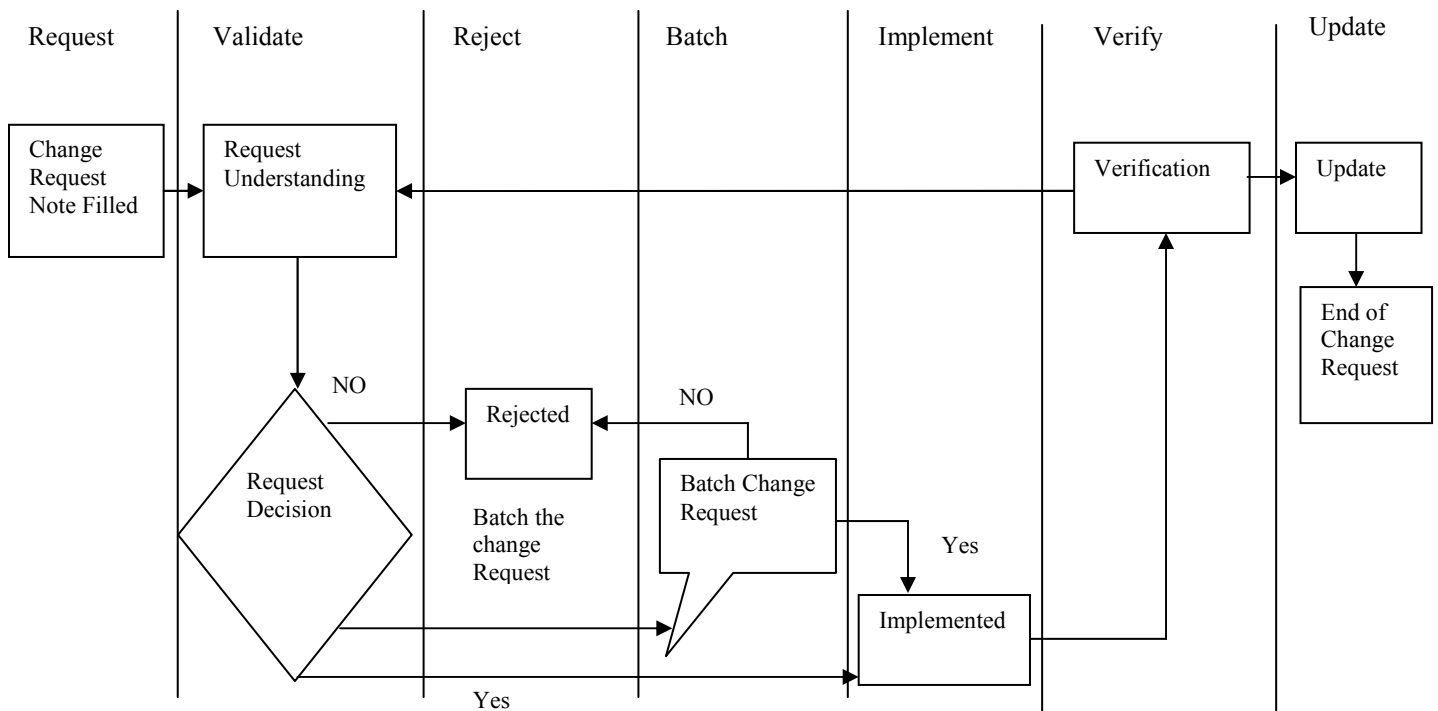


Figure 5. Proposed Process model for requirements change management in collocated software development

III. OBJECTIVE OF THE STUDY

In this work the main objective is to develop a process model for requirements change management in collocated software development. The proposed model will manage different requirement changes at different software development phases. This model is suggested to reduce the shortcomings of available models. It also presumes to reduce the failure rate of the software projects.

IV. METHODOLOGY

The information presented here is extracted from literature on requirements change management. Different forms of literature have been reviewed which include, requirements change management models [3, 10, 11], case studies [8] and requirements engineering books [11]. Most of the literature reports are real life experiences of requirements change management implementation. During the data extraction phase only that data is extracted which is related to requirements change management. During the review process, a SLR (Systematic Literature Review) was employed as suggested by an evidence-based software engineering paradigm [12]. According to Kitchenham et al. [12], there are three main phases of the SLR, i.e., planning the review, conducting the review and reporting the review. In the planning the review phase, the development of the selection criteria for the literature review is performed while conducting the review is related to data extraction and synthesis. The reporting the review phase includes a proficient way of report writing in order to describe the results from the literature.

V. PROPOSED REQUIREMENTS CHANGE MANAGEMENT MODEL

The proposed model is depicted in Fig. 5. The model is divided into seven core stages: Request, validate, reject, batch, Implement, verify and Update. Each of these stages

has specific activities which take place during the requirements change management process.

The first stage is the change request in the system as earlier discussed in Ince's Model [10]. The change request is expected from different stakeholders, i.e., project team members or customers. The first stage (request phase) contains all of the information about the change, e.g., complete explanation of the change, what the reasons for the change are, who has made the change request etc.

The next stage of the model is to validate, which is adopted from a Spiral like change management model and Ince's Model [10]. This stage contains two activities. The first activity is to understand the change request (e.g., to know whether the change request is to add some new features to the system or to remove any error from the system or to enhance any feature of the system). In this stage, the change request is evaluated through a number of checks. The impact of the change on budget, time and other system components is also evaluated in this stage. After understanding the change, the next activity is to make the decision about the change; this is whether the change should be accepted, rejected or batched.

The third stage after the validation stage is the "reject". If the requested change is not approved in the validation stage, then it will be sent to the reject stage. This phase contains all rejected change requests.

The fourth stage is batch, the same as discussed by Ince's Model [10]. After taking the decision about the change request in the validation stage, some change requests will be batched. The batch change requests could be implemented in the future but at that moment could not be implemented due to some limitations. There are two possibilities regarding batch change requests; if the limitations are relaxed within a given time, then they will be implemented otherwise they will be sent back to the reject phase.

The fifth stage is “implement”. In this stage, all the accepted changes are implemented in the system.

The next stage after implementation is “verify”. In this stage those changes which are accepted and implemented are sent forward to the verify stage. The verify stage is adopted from Olsen’s change management model [11]. In verification it is conformed whether the software is going to fulfill its specification after the change. Those changes for which verification is not successful will be sent back to the validate phase for more understanding.

The last stage is “Update”. All of the system documents will be updated with the change made. All the system stakeholders will be informed about the updates. Finally, the software will be released with the new updated changes.

VI. RESULTS AND DISCUSSIONS

This study focuses on the development of the software requirements change management model. The purpose of the research is to overcome the shortcomings found in the existing models through the proposed requirements change management model. It was found during the literature review that the existing models cannot handle the requirements change management due to the absence of some of the important activities. The similarities and the differences of the proposed model with the existing models are based on the activities as shown in Table 1.

	Bhatti Change Management Process	Spiral Model	Ince's Model	Olsen's Model	Propose Model
Change Request	√	√	√	X	√
Validate	√	√	√	√	√
Reject	√	√	√	√	√
Batch	X	X	√	X	√
Implement	√	√	√	√	√
Verify	X	X	X	√	√
Update	√	√	√	√	√

Table 1. Comparison of the proposed model with the existing models in literature

The comparison is based on the following scope.

- Which activities are missing in the existing models?
- Which missing activities are covered by the proposed model?

The symbol “√” and “X” shows the presence and absence of the activities in the models respectively. The models are compared on the base of the change request, validate, reject, batch, implement, verify and update activities.

Table 1 show that the Bhatti Change Management Process lacks two activities which are batch and verify. Due to the absence of the batch activity, the model cannot present any

information about those changes which are not implemented immediately but can be implemented after some period of time. Similarly, the model lacks the verify activity and the model does not provide clear information as to whether the implemented change is working correctly or not.

In the spiral like change management model, no verification activity is presented. So it is not possible to recognize whether the change which is implemented is working accurately or not. Similarly, there is no batch activity and the model completely ignores those change requests which cannot be implemented immediately.

In Ince’s model, it is found that no verification activity is referred to verify that the implemented change is accepted or not.

Olsen’s Model lacks the basic activity, i.e., the change request, it is necessary to get the information about the stakeholder who requested the change. There is no mechanism to record the information about the delayed changes.

The proposed change management model tries to cover all of those activities which are missing in the existing models. In Table 1, it is shown that the proposed model has all the activities on the basis of which the models were compared. The first activity is the change request in which the request for the change occurs. In this phase, complete information about the change request is gathered. In the validate phase the legitimacy of the change request is checked and an understanding of the change request is attempted to be gained. The next phase is the reject phase. This phase contains all of those change requests which are rejected after checking their validity. The batch phase accommodates the change requests which are not implemented immediately but can be implemented after some period of time. Those change requests which are accepted are sent forward to the implementation phase where these change requests are implemented. After implementation, the verification of the change request occurs to check whether these implemented changes are working properly or not. At the last step, the system is updated with the change.

VII. CONCLUSION

Requirements change is necessary and can occur at any phase of the software development. Managing the requirements change is a difficult and tough job. However, proper change management can help to perform change management with ease and lead to the completion of the project within time allotted and on budget. Different requirements change management models can be found in literature but they have some shortcomings as discussed earlier. The proposed requirements change management is developed to overcome the shortcomings of the existing requirements change management models.

VIII. FUTURE WORK

In the future, the model would be implemented and validated on real software development projects. The limitations (if any) would be identified and the proposed model would be improved after its implementation.

IX. REFERENCES

- [1] M. Wasim Bhatti, *et al.*, "An investigation of changing requirements with respect to development phases of a software project," in *Computer Information Systems and Industrial Management Applications (CISIM)*, 2010 International Conference on, 2010, pp. 323-327.
- [2] L. Lianzhong, *et al.*, "Process-related software requirements management," in *Computer Science and Information Technology (ICCSIT)*, 2010 3rd IEEE International Conference on, 2010, pp. 361-365.
- [3] M. W. Bhatti, *et al.*, "A methodology to manage the changing requirements of a software project," in *Computer Information Systems and Industrial Management Applications (CISIM)*, 2010 International Conference on, 2010, pp. 319-322.
- [4] J. Zhu, *et al.*, "The Requirements Change Analysis for Different Level Users," in *Intelligent Information Technology Application Workshops, 2008. IITAW '08. International Symposium on*, 2008, pp. 987-989.
- [5] G. E. Stark, *et al.*, "An examination of the effects of requirements changes on software maintenance releases," *Journal of Software Maintenance*, vol. 11, pp. 293-309, 1999.
- [6] S. McGee and D. Greer, "A Software Requirements Change Source Taxonomy," in *Software Engineering Advances, 2009. ICSEA '09. Fourth International Conference on*, 2009, pp. 51-58.
- [7] S. Ramzan and N. Ikram, "Making Decision in Requirement Change Management," in *Information and Communication Technologies, 2005. ICICT 2005. First International Conference on*, 2005, pp. 309-312.
- [8] W. Hussain, "Requirements Change Management in Global Software Development: A Case Study in Pakistan" 2010.
- [9] S. Ramzan and N. Ikram, "Requirement Change Management Process Models: Activities, Artifacts and Roles," in *Multitopic Conference, 2006. INMIC '06. IEEE*, 2006, pp. 219-223.
- [10] M. Makarainen, "Software change management process in the development of embedded software," Master, Faculty of science, University of Oulu, 2000.
- [11] J. L. Maté and A. Silva, *Requirements engineering for sociotechnical systems*: Information Science Pub., 2005.
- [12] B. Kitchenham, *et al.*, "Systematic literature reviews in software engineering - A tertiary study," *Inf. Softw. Technol.*, vol. 52, pp. 792-805, 2010.